

Deliverables 3: Reflection and Implications

Prepared by: Macpherson Aghah

Course: IT316 **Date:** June 12, 2026

Comparative Insights and Core Lessons

The Steps I Took

I began my reflection by mapping the functional capabilities of the Metasploit web delivery module against the other two platforms I researched, which were PowerShell Empire and Cobalt Strike. I focused specifically on how each framework handles payload delivery and evasion during the critical moments after an initial intrusion.

The Challenges I Faced

My primary challenge during this comparative review was looking past the massive differences in their user interfaces to see the underlying execution mechanics. Metasploit uses a standard console terminal, Empire relies on an API or the Starkiller interface, and Cobalt Strike utilizes a dedicated team server layout. It was easy to get distracted by how the tools look rather than focusing on how their agents interact with system memory.

How I Solved It

I solved this by tracking the exact execution footprint each tool leaves behind. I stripped away the interface layers and created a clear mental map of their operational specializations:

- **Metasploit Versatility:** The web delivery module stands out because it can dynamically shift between completely different payload languages like Python, PHP, and PowerShell depending on the target environment.
- **Empire Script Specialization:** Empire focuses heavily on maximizing living off the land techniques specifically within Windows environments, pushing script obfuscation to its absolute limits.
- **Cobalt Strike Traffic Control:** Cobalt Strike takes a different route by prioritizing highly customizable communication pathways through its unique Malleable C2 profiles, allowing agents to blend into legitimate corporate web traffic seamlessly.

Coding Practices and Architectural Design Patterns

The Steps I Took

I deeply analyzed the Ruby source code of the Metasploit module to isolate the exact programming patterns that allow it to remain highly efficient and stable.

The Challenges I Faced

When I first opened the source file, I was confused by how short the script actually was. I struggled to understand how a module with fewer than two hundred lines of unique code could simultaneously spin up a multithreaded web application server, parse incoming web headers, and format complex target commands.

How I Solved It

I solved this hurdle by tracing the object oriented inheritance lines within the Metasploit ecosystem. I discovered that the power of this module comes entirely from its use of mixin design patterns, specifically the inclusion of `HttpServer` and `CmdStager`.

Key Takeaway: The mixin architecture allows developers to reuse complex code libraries effortlessly. The unique script only handles the specific configuration details, while the heavy lifting of network socket management is safely handed off to trusted, prebuilt framework engines.

Defensive Implications and Behavioral Engineering

The Steps I Took

I pivot my focus toward enterprise blue team engineering, analyzing how understanding this specific code execution path helps me build better detection strategies for corporate defense.

The Challenges I Faced

The core defensive challenge presented by memory only web delivery mechanisms is that they bypass traditional signature detection. Because the target utility streams the script directly into its volatile memory allocation pages, file on disk antivirus tools are completely blind to the attack. There is no physical file write event for a scanner to intercept and analyze.

How I Solved It

I solved this defensive visibility gap by shifting my tracking strategy from static file signatures to real time behavioral monitoring at both the host and network layers.

Conclusion

Analyzing the web delivery module provided me with a perfect bridge between offensive theory and defensive engineering. By learning exactly how an installer manipulates memory to evade traditional disk storage checks, I gained the specific insights needed to build resilient, behavior based enterprise defenses that keep corporate networks secure.